

Atividade 4 - Compressão, Representação e Descrição

Aluno: Vitor Fontenele de Oliveira Linhares - 1700778

Repositório

Imports

```
In [111... import numpy as np
import cv2
import matplotlib.pyplot as plt
from pathlib import Path
```

Setup

```
In [112... plt.rcParams['image.cmap'] = 'gray'

ARTEMIS_PATH = Path('artemis.jpg')
CRATERS_PATH = Path('craters_moon.jpg')
EARTH_SUN_PATH = Path('earth_and_sun.jpg')
SUN_PATH = Path('sun.jpg')
OUT_DIR = Path('outputs')

def load_image(path):
    path = Path(path)
    if not path.exists():
        raise FileNotFoundError(f'Imagem não encontrada: {path.resolve()}')
    bgr = cv2.imread(str(path), cv2.IMREAD_UNCHANGED)
    if bgr is None:
        raise ValueError(f'Não foi possível ler a imagem: {path.resolve()}')
    if bgr.ndim == 2:
        return bgr
```

```
    return cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB)

def to_gray(img):
    if img.ndim == 2: return img
    return (0.299 * img[:, :, 0] + 0.587 * img[:, :, 1] + 0.114 * img[:, :, 2]).astype(np.float32)

def to_bin(img, threshold=128):
    if img.ndim == 3:
        img = to_gray(img)
    return (img >= threshold).astype(np.uint8) * 255

def save_image(filename, img, out_dir=OUT_DIR):
    out_dir = Path(out_dir)
    out_dir.mkdir(parents=True, exist_ok=True)
    out_path = out_dir / filename
    arr = np.clip(img, 0, 255).astype(np.uint8)
    if arr.ndim == 3:
        arr = cv2.cvtColor(arr, cv2.COLOR_RGB2BGR)
    cv2.imwrite(str(out_path), arr)
    return out_path

def show_grid(images, titles, cols, figsize=None):
    n = len(images)
    rows = (n + cols - 1) // cols

    if figsize is None:
        figsize = (cols * 4, rows * 4)

    _, axes = plt.subplots(rows, cols, figsize=figsize)
    axes = axes.flatten() if n > 1 else [axes]

    for i in range(len(axes)):
        if i < n:
            axes[i].imshow(images[i], vmin=images[i].min(), vmax=images[i].max())
            axes[i].set_title(titles[i])
            axes[i].axis('off')

    plt.tight_layout()
    plt.show()
```

```
def initialize():
    artemis = load_image(ARTEMIS_PATH)
    earth_and_sun = load_image(EARTH_SUN_PATH)
    sun = load_image(SUN_PATH)

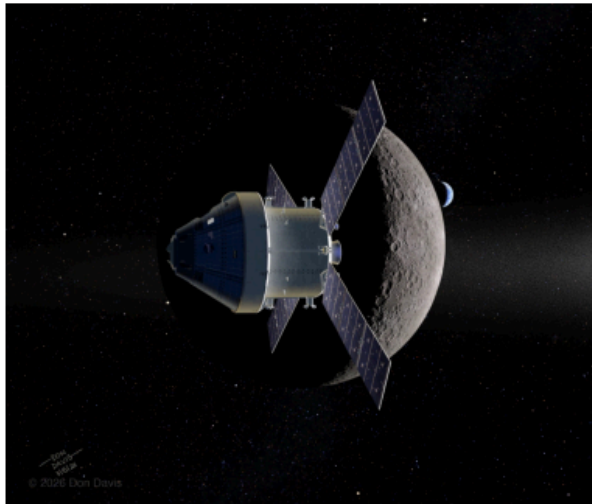
    artemis_gray = to_gray(artemis)
    earth_and_sun_bin = to_bin(earth_and_sun)
    sun_bin = to_bin(sun)

    save_image('artemis_gray.jpg', artemis_gray, OUT_DIR)
    save_image('earth_and_sun_bin.jpg', earth_and_sun_bin, OUT_DIR)
    save_image('sun_bin.jpg', sun_bin, OUT_DIR)

    show_grid(
        [artemis, earth_and_sun, sun, artemis_gray, earth_and_sun_bin, sun_bin],
        ['Artemis', 'Earth and Sun', 'Sun', 'Artemis Gray', 'Earth and Sun Bin', 'Sun Bin'],
        cols=3
    )

initialize()
```

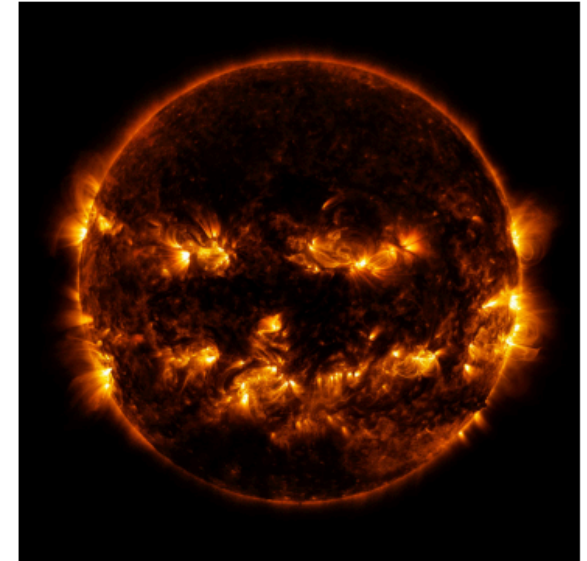
Artemis



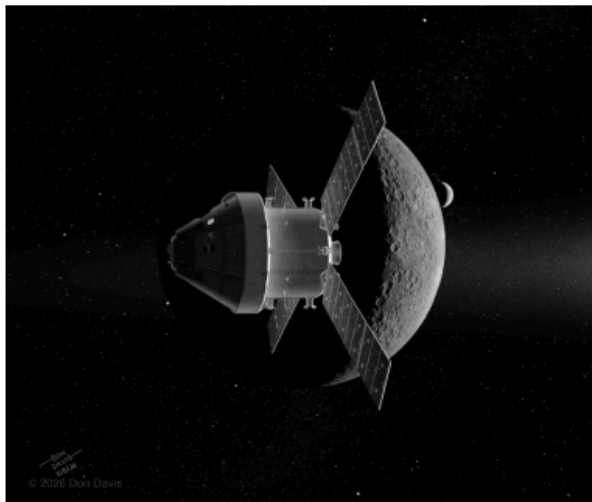
Earth and Sun



Sun



Artemis Gray



Earth and Sun Bin



Sun Bin



Questão 1

Operações Base

```
In [113... def erosion(img, kernel):
    h, w = img.shape
    pad = kernel // 2
    out_img = np.zeros_like(img)

    padded_img = np.pad(img, pad, mode='edge')
    expected_sum = kernel * kernel * 255

    for y in range(h):
        for x in range(w):
            window = padded_img[y:y + kernel, x:x + kernel]

            if np.sum(window) == expected_sum:
                out_img[y, x] = 255

    return out_img

def dilation(img, kernel):
    h, w = img.shape
    pad = kernel // 2
    out_img = np.zeros_like(img)

    padded_img = np.pad(img, pad, mode='edge')

    for y in range(h):
        for x in range(w):
            window = padded_img[y:y + kernel, x:x + kernel]

            if np.sum(window) > 0:
                out_img[y, x] = 255

    return out_img

def opening(img_bin, kernel_size):
    eroded = erosion(img_bin, kernel_size)
    return dilation(eroded, kernel_size)
```

```
def closing(img_bin, kernel_size):  
    dilated = dilation(img_bin, kernel_size)  
    return erosion(dilated, kernel_size)
```

Retornar Tupla da Morfologia

```
In [114... def process_morphology(img, kernel):  
    erosion_result = erosion(img, kernel)  
    dilation_result = dilation(img, kernel)  
    opening_result = opening(img, kernel)  
    closing_result = closing(img, kernel)  
  
    return erosion_result, dilation_result, opening_result, closing_result
```

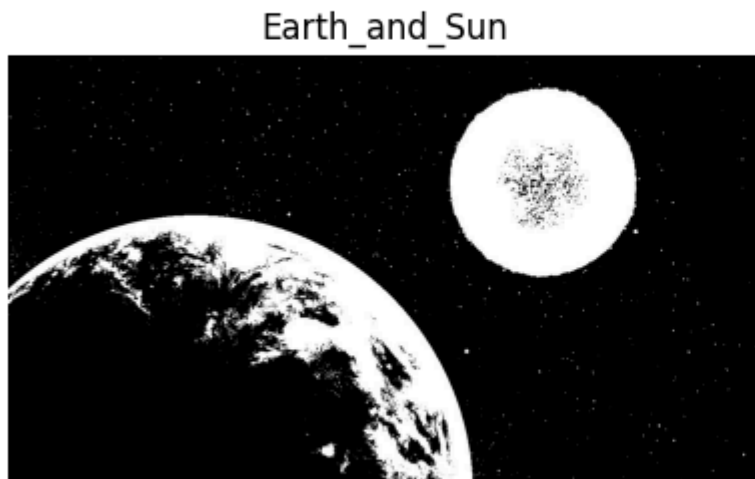
Rodando

```
In [115... images_bin = [  
    load_image(OUT_DIR / 'earth_and_sun_bin.jpg'),  
    load_image(OUT_DIR / 'sun_bin.jpg')  
]  
images_names = ["Earth_and_Sun", "Sun"]  
kernels = [3, 5, 15]  
  
show_grid(images_bin, images_names, cols=2)  
  
for idx in range(len(images_bin)):  
    img_bin = images_bin[idx]  
    name = images_names[idx]  
  
    for k_size in kernels:  
        print(f"{name} | Kernel: {k_size}x{k_size}")  
  
        eroded, dilated, opened, closed = process_morphology(img_bin, k_size)  
  
        titles = [  
            f"Erosão {k_size}x{k_size}",  
            f"Dilatação {k_size}x{k_size}",  
            f"Abertura {k_size}x{k_size}",
```

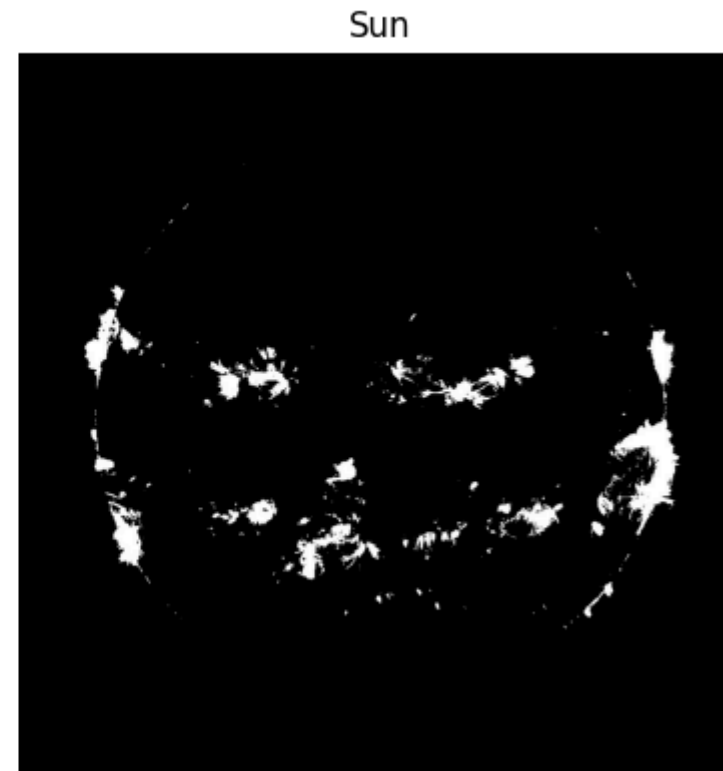
```
        f"Fechamento {k_size}x{k_size}"
    ]

    save_image(f"{name.lower()}_erosion_{k_size}x{k_size}.jpg", eroded, OUT_DIR)
    save_image(f"{name.lower()}_dilation_{k_size}x{k_size}.jpg", dilated, OUT_DIR)
    save_image(f"{name.lower()}_opening_{k_size}x{k_size}.jpg", opened, OUT_DIR)
    save_image(f"{name.lower()}_closing_{k_size}x{k_size}.jpg", closed, OUT_DIR)

    show_grid([eroded, dilated, opened, closed], titles, cols=4)
```



Earth_and_Sun | Kernel: 3x3





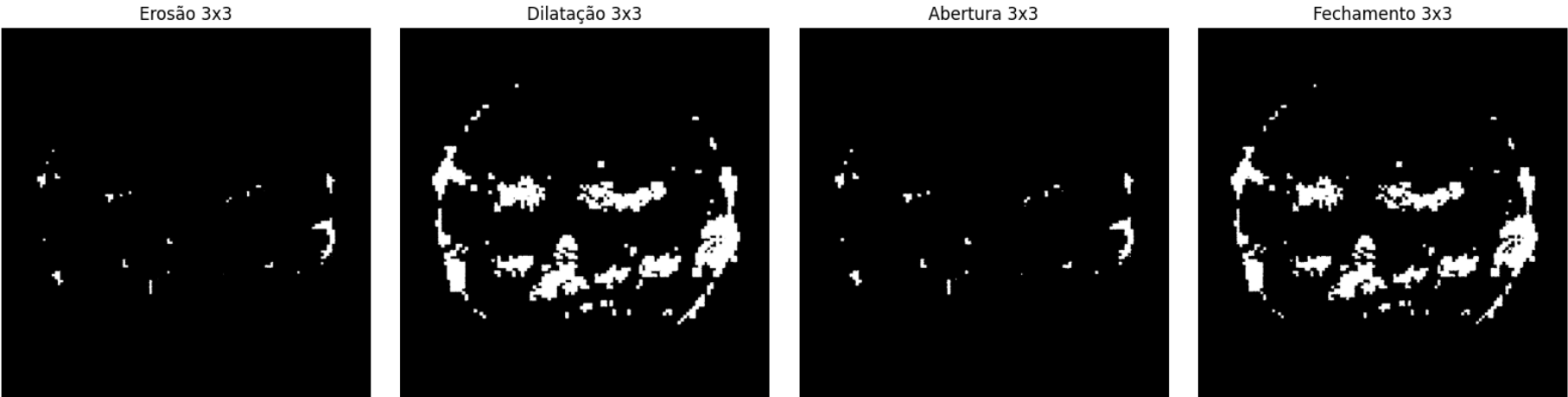
Earth_and_Sun | Kernel: 5x5



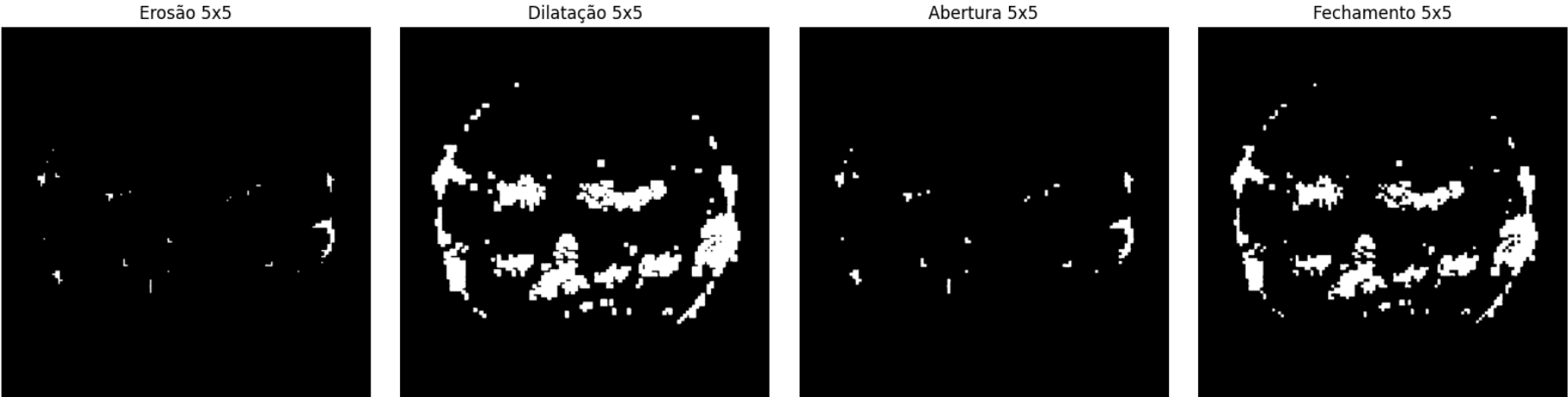
Earth_and_Sun | Kernel: 15x15



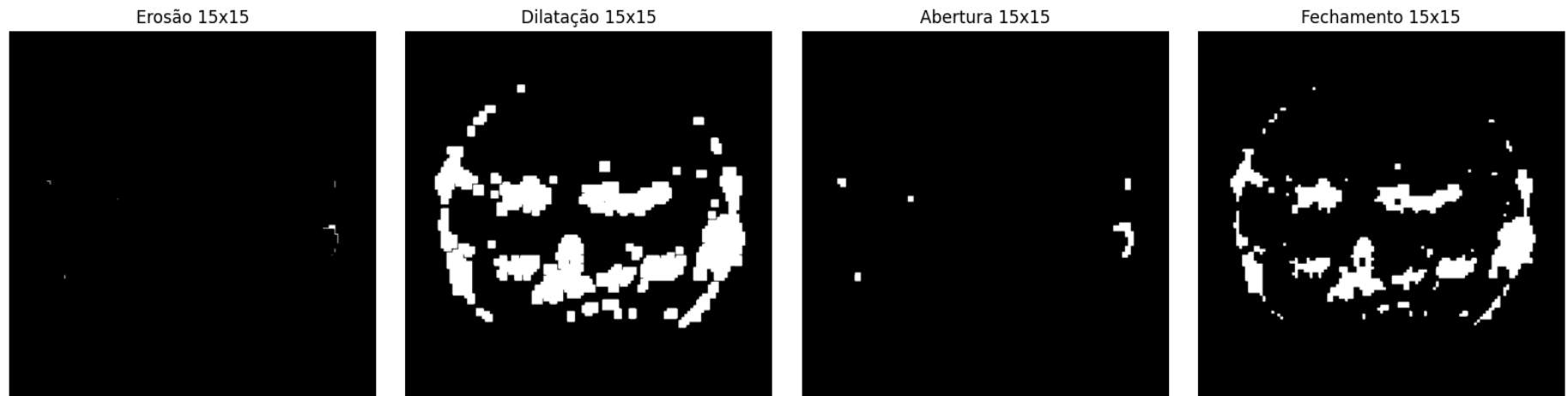
Sun | Kernel: 3x3



Sun | Kernel: 5x5



Sun | Kernel: 15x15



Análise

Erosão e Dilatação

- **Erosão:** Reduziu progressivamente a área dos corpos principais conforme o kernel aumentou. O efeito concentrou-se no encolhimento das massas principais, sem remoção de elementos isolados, como as estrelas eram discretas, no primeiro kernel elas já sumiram.
- **Dilatação:** Atuou de forma inversa, expandindo uniformemente as bordas das estruturas claras. Com o aumento da máscara, essa expansão distorceu as geometrias originais e tendeu a fundir contornos próximos.

Abertura e Fechamento

- **Abertura:** Eliminou detalhes finos e preservou apenas formas macroscópicas dominantes com o crescimento do kernel. Na máscara 15×15 , restaram apenas as silhuetas simplificadas em `Earth_and_Sun`, enquanto em `Sun` ficou apenas ruído.
- **Fechamento:** Atuou agregando estruturas e suavizando a figura. O operador incorporou irregularidades periféricas e eliminou falsos vales nas bordas do Sol e da Terra, conectando componentes próximos e tornando as superfícies mais contínuas, sem alterar destrutivamente a área dos corpos.

Questão 2

Convolução, Filtro Gaussiano e Sobel

```
In [116... def convolution(img_gray, kernel):
    h, w = img_gray.shape
    kh, kw = kernel.shape
    pad_h, pad_w = kh // 2, kw // 2

    kernel_flipped = kernel[::-1, ::-1]

    padded = np.pad(img_gray, ((pad_h, pad_h), (pad_w, pad_w)), mode='edge')
    out = np.zeros_like(img_gray, dtype=np.float32)

    for y in range(h):
        for x in range(w):
            window = padded[y:y + kh, x:x + kw]
            out[y, x] = np.sum(window * kernel_flipped)

    return out

def gaussian_kernel(size, sigma):
    ax = np.linspace(-(size // 2), size // 2, size)
    xx, yy = np.meshgrid(ax, ax)
    kernel = np.exp(-(xx**2 + yy**2) / (2.0 * sigma**2))
    return kernel / np.sum(kernel)

def sobel(img_blur):
    sobel_x = np.array([[ -1,  0,  1],
                        [ -2,  0,  2],
                        [ -1,  0,  1]], dtype=np.float32)

    sobel_y = np.array([[ -1, -2, -1],
                        [  0,  0,  0],
                        [  1,  2,  1]], dtype=np.float32)

    gx = convolution(img_blur, sobel_x)
    gy = convolution(img_blur, sobel_y)

    magnitude = np.sqrt(gx**2 + gy**2)
    orientation = np.arctan2(gy, gx)

    return magnitude, orientation
```

Hog

```

In [117... def hog(magnitude, orientation, cell_size=8, num_bins=9):
    h, w = magnitude.shape
    num_cells_y = h // cell_size
    num_cells_x = w // cell_size

    hog_cells = np.zeros((num_cells_y, num_cells_x, num_bins), dtype=np.float32)

    orientation_deg = np.degrees(orientation) % 180
    bin_width = 180.0 / num_bins

    for cy in range(num_cells_y):
        for cx in range(num_cells_x):
            mag_tile = magnitude[cy*cell_size:(cy+1)*cell_size, cx*cell_size:(cx+1)*cell_size]
            ang_tile = orientation_deg[cy*cell_size:(cy+1)*cell_size, cx*cell_size:(cx+1)*cell_size]

            for y in range(cell_size):
                for x in range(cell_size):
                    pix_mag = mag_tile[y, x]
                    pix_ang = ang_tile[y, x]

                    if pix_mag > 150.0:
                        bin_idx = int(pix_ang // bin_width) % num_bins
                        hog_cells[cy, cx, bin_idx] += pix_mag

    centers_x = []
    centers_y = []
    u_dirs = []
    v_dirs = []

    half_cell = cell_size // 2
    bins_angles = np.linspace(0, np.pi, num_bins, endpoint=False) + (np.pi / (2 * num_bins))

    for cy in range(num_cells_y):
        for cx in range(num_cells_x):
            hist = hog_cells[cy, cx, :]

            if np.max(hist) == 0:

```

```

        continue

    max_bin = np.argmax(hist)
    angle = bins_angles[max_bin]

    center_y = cy * cell_size + half_cell
    center_x = cx * cell_size + half_cell

    centers_x.append(center_x)
    centers_y.append(center_y)

    u_dirs.append(np.cos(angle))
    v_dirs.append(np.sin(angle))

    return (np.array(centers_x), np.array(centers_y), np.array(u_dirs), np.array(v_dirs))

```

Função Orquestradora

```

In [118... def process_canny_and_hog(img_gray, gaussian_size=5, sigma=1.4, cell_size=8, num_bins=9):
    kernel_g = gaussian_kernel(gaussian_size, sigma)
    blurred = convolution(img_gray, kernel_g)
    magnitude, orientation = sobel(blurred)
    hog_vectors = hog(magnitude, orientation, cell_size, num_bins)

    return blurred, magnitude, hog_vectors

```

Rodando

```

In [119... artemis_gray = to_gray(load_image(OUT_DIR / 'artemis_gray.jpg'))
blurred, magnitude, hog_vectors = process_canny_and_hog(artemis_gray)

save_image('artemis_blurred.jpg', blurred)
save_image('artemis_magnitude.jpg', magnitude)

hog_img = np.zeros_like(magnitude, dtype=np.uint8)

cx, cy, u, v = hog_vectors
h_max, w_max = hog_img.shape

```

```
for x, y, dx, dy in zip(cx, cy, u, v):
    for t in range(-3, 4):
        px = int(x + t * dx)
        py = int(y + t * dy)

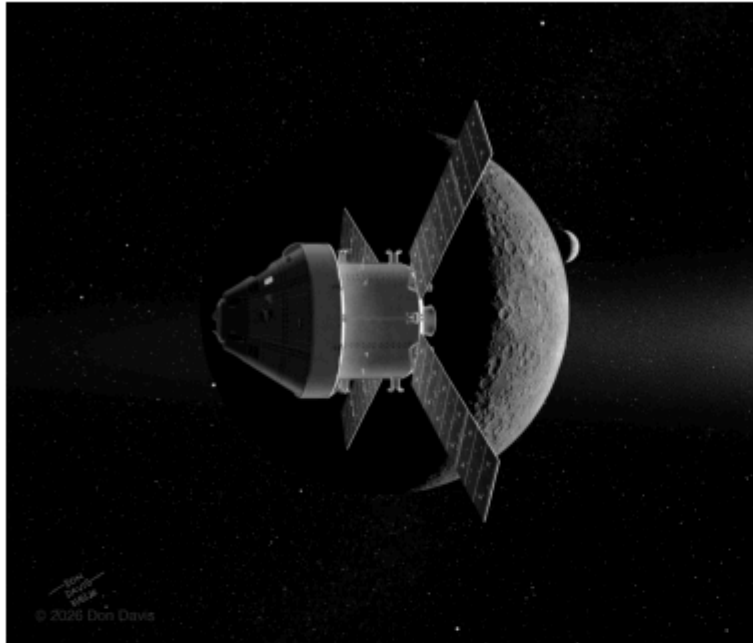
        if 0 <= px < w_max and 0 <= py < h_max:
            hog_img[py, px] = 255

save_image('artemis_hog.jpg', hog_img)

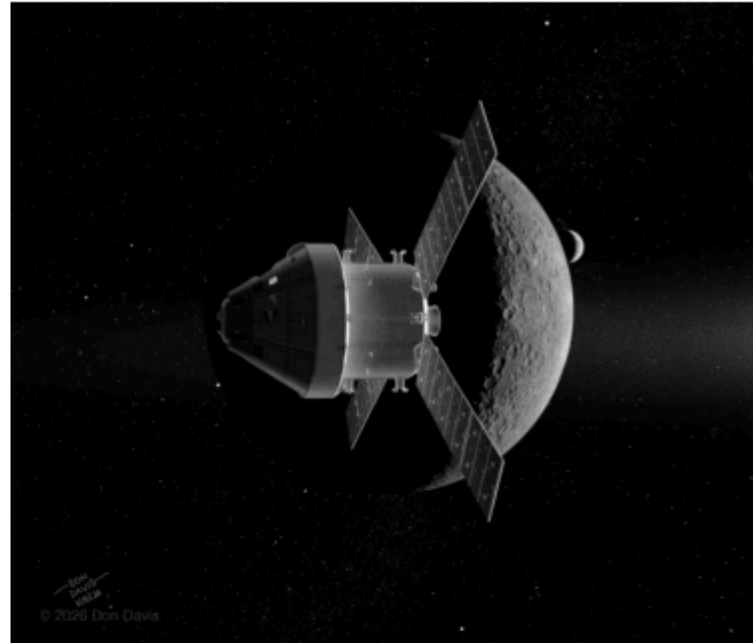
grid_images = [artemis_gray, blurred, magnitude, hog_img]
grid_titles = [
    "0. Artemis Gray",
    "1. Blur Gaussiano",
    "2. Sobel",
    "3. HOG"
]

show_grid(grid_images, grid_titles, cols=2)
```

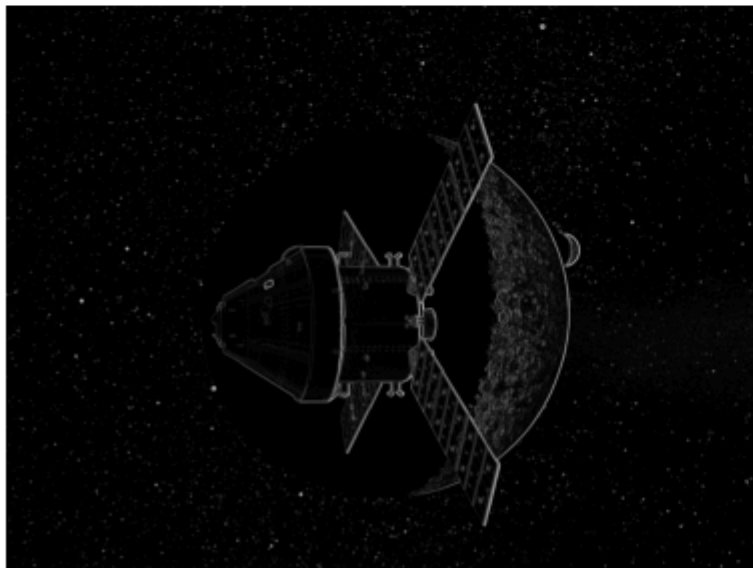
0. Artemis Gray



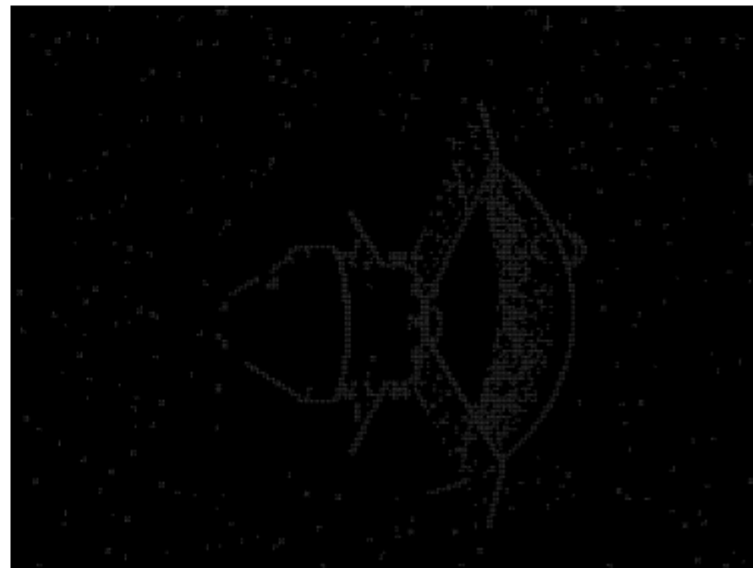
1. Blur Gaussiano



2. Sobel



3. HOG





Análise

O gradiente de Sobel mapeou as transições locais de intensidade, acendendo os contornos, a parte iluminada da Lua e os pontos correspondentes às estrelas de fundo. Na extração do HOG, a natureza densamente estrelada do cenário espacial inicialmente contaminou o descritor sob limiares baixos, visto que cada estrela gerava variações locais que ativavam erroneamente os histogramas das células (foi necessário colocar o limiar da magnitude em **150.0**). Contudo, com a aplicação do limiar de magnitude ajustado, o ruído foi reduzido, permitindo que os vetores dominantes se alinhassem ao longo das bordas principais da Artemis e da silhueta da lua. Esse experimento foi interessante, pois mostra as dificuldades e a eficiência do algoritmo que é capaz de reconhecer os vetores mesmo diante de um fundo altamente denso.

Questão 3

Funções de Preparação

```
In [120... def distance_transform(binary_img):
    h, w = binary_img.shape
    dist = np.where(binary_img == 0, 0, np.inf)

    for r in range(h):
        for c in range(w):
            if dist[r, c] > 0:
                top = dist[r - 1, c] + 1 if r > 0 else np.inf
                left = dist[r, c - 1] + 1 if c > 0 else np.inf
                dist[r, c] = min(dist[r, c], top, left)

    for r in range(h - 1, -1, -1):
        for c in range(w - 1, -1, -1):
            if dist[r, c] > 0:
                bottom = dist[r + 1, c] + 1 if r < h - 1 else np.inf
                right = dist[r, c + 1] + 1 if c < w - 1 else np.inf
                dist[r, c] = min(dist[r, c], bottom, right)

    return dist
```



```
def preprocess_morphology_and_distance(binary_img, kernel):
    cleaned_bin = opening(binary_img, kernel)
    dist_map = distance_transform(cleaned_bin)

    return cleaned_bin, dist_map
```

Marcadores

```
In [121]... def _bfs(r, c, peaks, markers, visited, label, h, w):
    queue = [(r, c)]
    visited[r, c] = True

    while queue:
        curr_r, curr_c = queue.pop(0)
        markers[curr_r, curr_c] = label

        for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
            nr, nc = curr_r + dr, curr_c + dc

            if 0 <= nr < h and 0 <= nc < w:
                if peaks[nr, nc] == 255 and not visited[nr, nc]:
                    visited[nr, nc] = True
                    queue.append((nr, nc))

def extract_and_label_markers(dist_map, threshold):
    h, w = dist_map.shape
    max_dist = np.max(dist_map)

    if max_dist == 0:
        return np.zeros((h, w), dtype=np.int32)

    peaks = np.where(dist_map > (max_dist * threshold), 255, 0).astype(np.uint8)

    markers = np.zeros((h, w), dtype=np.int32)
    visited = np.zeros((h, w), dtype=bool)
    label = 1

    for r in range(h):
        for c in range(w):
```

```

        if peaks[r, c] == 255 and not visited[r, c]:
            _bfs(r, c, peaks, markers, visited, label, h, w)
            label += 1

    return markers

```

Watershed

```

In [122... def watershed(dist_map, markers):
    h, w = dist_map.shape
    labels = markers.copy()

    topo_surface = (np.max(dist_map) - dist_map).astype(np.int32)

    init_r, init_c = np.where(labels > 0)
    boundary_pixels = []
    in_queue = np.zeros((h, w), dtype=bool)

    for r, c in zip(init_r, init_c):
        in_queue[r, c] = True
        for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
            nr, nc = r + dr, c + dc
            if 0 <= nr < h and 0 <= nc < w:
                if labels[nr, nc] == 0 and not in_queue[nr, nc]:
                    in_queue[nr, nc] = True
                    boundary_pixels.append((topo_surface[nr, nc], nr, nc))

    boundary_pixels.sort(key=lambda x: x[0])

    while boundary_pixels:
        _, curr_r, curr_c = boundary_pixels.pop(0)

        neighbor_labels = []
        for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
            nr, nc = curr_r + dr, curr_c + dc
            if 0 <= nr < h and 0 <= nc < w and labels[nr, nc] > 0:
                neighbor_labels.append(labels[nr, nc])

        if neighbor_labels:
            unique_labels = np.unique(neighbor_labels)

```

```

    if len(unique_labels) == 1:
        labels[curr_r, curr_c] = unique_labels[0]
    else:
        labels[curr_r, curr_c] = -1

    for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
        nr, nc = curr_r + dr, curr_c + dc
        if 0 <= nr < h and 0 <= nc < w:
            if labels[nr, nc] == 0 and not in_queue[nr, nc]:
                in_queue[nr, nc] = True
                item = (topo_surface[nr, nc], nr, nc)

                idx = 0
                while idx < len(boundary_pixels) and boundary_pixels[idx][0] < item[0]:
                    idx += 1
                boundary_pixels.insert(idx, item)

    return labels

```

Orquestração

```

In [123... def generate_label_color_image(labels):
    h, w = labels.shape
    color_img = np.zeros((h, w, 3), dtype=np.uint8)

    np.random.seed(42)
    max_label = np.max(labels)

    colors = {0: [0, 0, 0], -1: [255, 0, 0]}
    for i in range(1, max_label + 1):
        colors[i] = list(np.random.randint(0, 255, size=3))

    for r in range(h):
        for c in range(w):
            lbl = labels[r, c]
            if lbl in colors:
                color_img[r, c] = colors[lbl]
            else:
                color_img[r, c] = [255, 255, 255]

```

```

    return color_img

def watershed_pipeline(binary_img, kernel_size, threshold_ratio):
    cleaned_bin, dist_map = preprocess_morphology_and_distance(binary_img, kernel_size)
    markers = extract_and_label_markers(dist_map, threshold_ratio)
    labels = watershed(dist_map, markers)
    colored_output = generate_label_color_image(labels)

    return cleaned_bin, dist_map, markers, labels, colored_output

```

Rodando

```

In [126... kernel = 5
threshold = 0.4

craters_baseline = to_bin(load_image(CRATERS_PATH))
craters_closed = closing(craters_baseline, kernel)

cleaned_bin_raw, dist_map_raw, markers_raw, _, colored_output_raw = watershed_pipeline(
    craters_baseline,
    kernel_size=kernel,
    threshold_ratio=threshold
)

cleaned_bin_cls, dist_map_cls, markers_cls, _, colored_output_cls = watershed_pipeline(
    craters_closed,
    kernel_size=kernel,
    threshold_ratio=threshold
)

dist_raw_norm = ((dist_map_raw - np.min(dist_map_raw)) / (np.max(dist_map_raw) - np.min(dist_map_raw)) * 255).astype(np.uint8)
markers_raw_norm = np.where(markers_raw > 0, 255, 0).astype(np.uint8)

dist_cls_norm = ((dist_map_cls - np.min(dist_map_cls)) / (np.max(dist_map_cls) - np.min(dist_map_cls)) * 255).astype(np.uint8)
markers_cls_norm = np.where(markers_cls > 0, 255, 0).astype(np.uint8)

grid_images = [
    craters_baseline, cleaned_bin_raw, dist_raw_norm, markers_raw_norm, colored_output_raw,
    craters_closed, cleaned_bin_cls, dist_cls_norm, markers_cls_norm, colored_output_cls
]

```

```

]

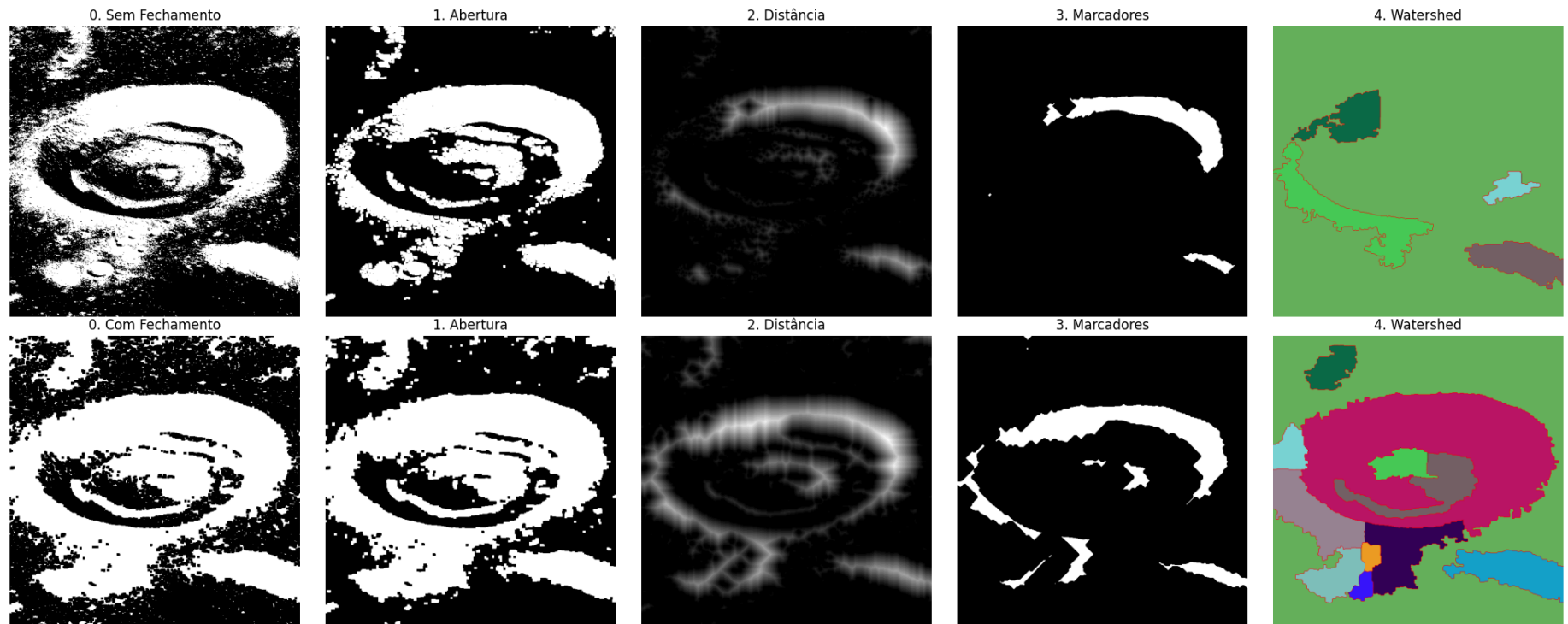
grid_titles = [
    "0. Sem Fechamento", "1. Abertura", "2. Distância", "3. Marcadores", "4. Watershed",
    "0. Com Fechamento", "1. Abertura", "2. Distância", "3. Marcadores", "4. Watershed"
]

save_image('craters_distance_baseline.jpg', dist_raw_norm)
save_image('craters_markers_baseline.jpg', markers_raw_norm)
save_image('craters_watershed_baseline.jpg', colored_output_raw)

save_image('craters_distance_closed.jpg', dist_cls_norm)
save_image('craters_markers_closed.jpg', markers_cls_norm)
save_image('craters_watershed_output.jpg', colored_output_cls)

show_grid(grid_images, grid_titles, cols=5)

```



Análise

A análise comparativa entre a pipeline com e sem o pré-processamento (operação morfológica defechamento na imagem binarizada) evidencia que a inclusão dessa operação é fundamental para o sucesso da segmentação por Watershed, corrigindo a entrada, sem isso os marcadores falham, o que compromete o Watershed. Ao transformar o anel das crateras em uma massa sólida, conferimos ao Watershed a capacidade de reconhecer as bordas reais com alta precisão e separar regiões adjacentes, mapeando, inclusive, a subcratera menor (região verde) inserida no interior da estrutura principal (região rosa).